

Course: **Object-Oriented Programming (D)**

18-April-2019

(Spring 2019)Resource Person: **Zunaina Islam****ASSIGNMENT–1 (Double Dimensional
Arrays, Sorting and Search)**

Total Points: 20**Submission Due: Wednesday April 25, 2019**

Instructions: Please Read Carefully!

- This is an individual assignment. Everyone is expected to complete the given assignment on their own, without seeking any help from any website or any other individual. There will be strict penalties for any work found copied from any source and the university policy on plagiarism will be strictly enforced.
 - You are expected to submit this assignment as:
 - Create a single **.java** file solution of the assignment question. The name of the **.java** file should be as mentioned in the assignment question.
 - Then create a single **.zip** file containing you solution **.java** file. This zip file must be named as rollnumber.zip for example 13130013.zip
 - Assignment is to be submitted via email at 14137017@gift.edu.pk
 - Email Subject should be **CS-239 OOP A1 Submission**.
 - Submit your assignment on or before due date. **No late submissions will be possible.**
-

Program Name: StudentList.java**[20]**

You are being asked to write a program which will keep the list of Students. The program will keep record for the Student's **Name**, **Roll number**, and **percentage**. You are required to use the **two-dimensional String array** for this purpose.

The array would have **n rows and 3 columns** where **n** is the size entered by user. The first, second, and third columns would be used for storing a student's **name**, **roll number** and **percentage** respectively. We will also treat **name** and **percentage** as a **String**. This array should be of this form:

Alia	151380014	87
Ahmed	182300012	45
Sheharyar	191980023	76

The program when run should do the following in sequence:

1. Ask the user for the **size** of the array, that is number of rows, and validate the size to ensure that it is greater than or equal to 5 and less than or equal to 10.
2. Display the menu as follows:
 1. Press 0 to Exit
 2. Press 1 to Add a New Student
 3. Press 2 to Search by Student's Name
 4. Press 3 to Search by Student's Roll number
 5. Press 4 to Sort by Student's Roll number
 6. Press 5 to Sort by Student's Percentage
 7. Press 6 to Display the Student with highest Percentage
 8. Press 7 to Display All Students
3. The choice entered by user must be between **0** to **7**. If the user enters any other number, ask them to enter the choice again.
4. The program should display the menu after every selection. For example, if user press **4**, the program should display the menu again after sorting the Students by Roll number.
5. There should be a method to perform each of the tasks above.

The details for each menu item are as follows:

- **Add a New Student**

Asks the user to enter the **name**, **Roll number**, and **percentage** for the new student, then add the student into the array.

Hint: You need to keep the record that how many students have been added to the array. For this purpose, you can use an integer variable which will work as a *counter*.

You may use the following header for this method:

```
static boolean add(String[][] students, int currentSize)
```

Note that the return type of the method is **boolean**, it means that if **currentSize** is less than the length of the array, then a new student can be added and the method should return **true** after adding the student to the array, or it will return **false** if the array is full.

You should display "**The list is full. No more Students can be added.**", if the list gets full.

- **Search by Student's Name**

Ask the user for the (name) he/she wants to search for, then call the method which searches the array for the Students's name and return the index where the name of the student is found. Returns -1 otherwise.

Hint: The **name** is stored in the **first** column, so you only need to search in every first column of the array only.

You may use the following header for this method:

```
int searchByName(String[][] students, int currentSize,
String key)
```

- **Search by Student's Roll number**

Ask the user for the rollnumber he/she wants to search for, then call the method which searches the array for the Student's roll number and return the index where the rollnumber of the student is found. Returns -1 otherwise.

Hint: The **rollnumber** is stored in the **second** column, so you only need to search in every second column of the array only.

You may use the following header for this method:

```
int searchByRollnumber(String[][] students, int
currentSize, String key)
```

- **Sort by Student's Rollnumber**

Sort the array by the **rollnumber** in *ascending order*.

NOTE: Sort by rollnumber does not mean that you sort the rollnumbers only, the name and percentages associated with each rollnumber must also be sorted. For example, consider the following list of students:

Sheharyar	191980023	76
Alia	151380014	87
Ahmed	182380023	45

When we sort this list by rollnumber, it will be changed to the following:

Alia	151380014	87
Ahmed	182300012	45
Sheharyar	191980023	76

- **Display All Students**

Displays all the students as following:

Name	Roll number	Percentage
Sheharyar	191980023	76
Alia	151380014	87
Ahmed	182380023	45

You may use the following header for this method:

```
static void displayAllStudents(String[][] students, int  
currentSize)
```

Useful String Methods:

You may find the following **String** methods useful. They will help you in searching and sorting.

NOTE: You can see the code in the **StringMethodExamples.java** file for examples.

- ***equals*** method

The String ***equals*** method takes another String value as an argument and compares both Strings (caller and argument) based on their Unicode values. It returns **true** if the caller String is equal to argument String, or **false** otherwise.

For example:

```
String str = "Java";  
boolean result = str.equals("Horse");
```

The value of the result will be **false** because "Java" is not equal to "Horse".

Take another example:

```
String str = "Java";  
boolean result = str.equals("Java");
```

The value of the result will be **true** because "Java" is equal to "Java".

- ***compareTo*** method

The String ***compareTo*** method takes another String value as an argument and compares both Strings (caller and argument) based on their Unicode values. The method does the following:

Returns 0 if the values of both caller and argument String objects is same.

Returns a positive value if the value of caller String object is greater than the value of argument String object.

Returns a negative value if the value of caller String object is less than the value of argument String object.

Examples:

```
String s1 = "java";  
String s2 = "java";  
String s3 = "nava";  
String s4 = "jasa";  
  
//Returns 0, because both are equal  
int result = s1.compareTo(s2);  
System.out.println(result);  
  
//Returns -4, because "j" is 4 times lower than  
"n" result = s1.compareTo(s3);  
System.out.println(result);  
  
//Returns 3 because "v" is 3 times greater than  
"s" result = s1.compareTo(s4);  
System.out.println(result);
```